# *Elastic Container Service*

# *AWS ECS Deployment with Application Load Balancer*

## 1. Task / Question

Question: Deploy a Docker-based hiring application on Amazon ECS using AWS Fargate and an Application Load Balancer. Make the application accessible through a browser and configure it to run across multiple Availability Zones.

## 2. Purpose of the task

The purpose is to learn how to deploy and run a containerized application on AWS without managing servers manually.

We use:

- Amazon ECS to manage containers.
- AWS Fargate to run containers without managing EC2 servers.
- Application Load Balancer (ALB) to distribute incoming traffic.
- Target Group to register tasks and check their health.
- Availability Zones to improve application availability.

## 3. How does this help in real time?

Imagine a company has a hiring website that applicants use to submit applications.

If thousands of people access the website, one container might not be enough. ECS can run multiple copies of the application, and the load balancer distributes requests among healthy copies.

If one task becomes unhealthy, the load balancer can stop sending traffic to that task. ECS can also replace failed tasks, helping keep the application available.

## 4. AWS services used

Amazon ECS

Manages the application's container tasks.

AWS Fargate

Runs containers without managing EC2 servers.

Application Load Balancer

Receives browser requests on HTTP port 80 and forwards them to healthy tasks.

Amazon VPC

Provides the network, subnets, and security groups for the application.

# 5. Steps we performed

## Step 1 — Open Amazon ECS

- Open the AWS Management Console.
- Select the Europe (Stockholm) — `eu-north-1` region.
- Open Amazon ECS.

Purpose: ECS is the service we use to deploy and manage containers

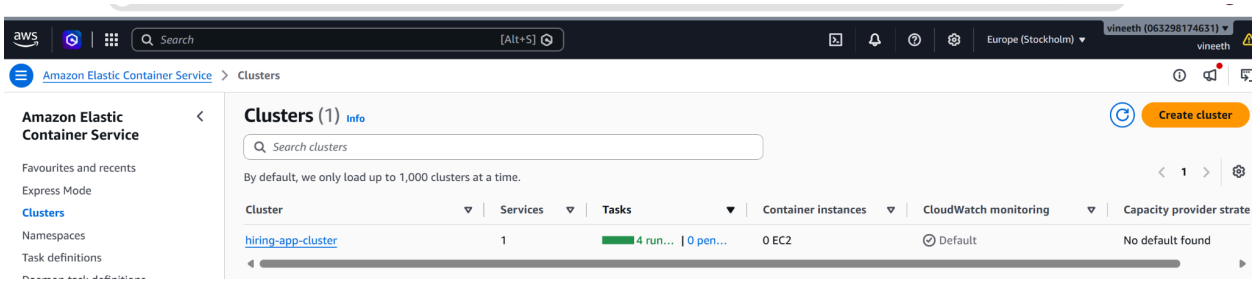
## Step 2 — Create the ECS cluster

We used the cluster:

`hiring-app-cluster`

- Created or selected the ECS cluster.
- Used AWS Fargate as the compute option.

Purpose: A cluster is the logical place where ECS manages our application services and tasks.




## Step 3 — Select the task definition

We used:

`hiring-app-task:2`

The task definition contains the instructions ECS needs to run the container, including the container image, CPU, memory, and port settings.

Our application container was named `hiring-app` and used port `8080`.

Purpose: The task definition tells ECS what application to run and how to run it.

## Step 4 — Create the ECS service

We configured the service in `hiring-app-cluster`.

- Launch type: AWS Fargate
- Scheduling strategy: Replica
- Desired count: initially set to 1, then increased to 6
- Availability Zone re-balancing: enabled

Purpose: An ECS service maintains the desired number of running tasks and replaces tasks when necessary.

Important: The service currently has 4 desired and running tasks. The attempt to reach 6 failed because the Fargate vCPU quota was reached.

## Step 5 — Configure networking

We used the existing VPC:

`vpc-04069a29bd5e93924`

We configured the service networking with subnets in multiple Availability Zones.

Purpose: Subnets determine where the tasks can run. Using multiple Availability Zones helps distribute tasks across separate AWS locations within the region.

## Step 6 — Create the Application Load Balancer

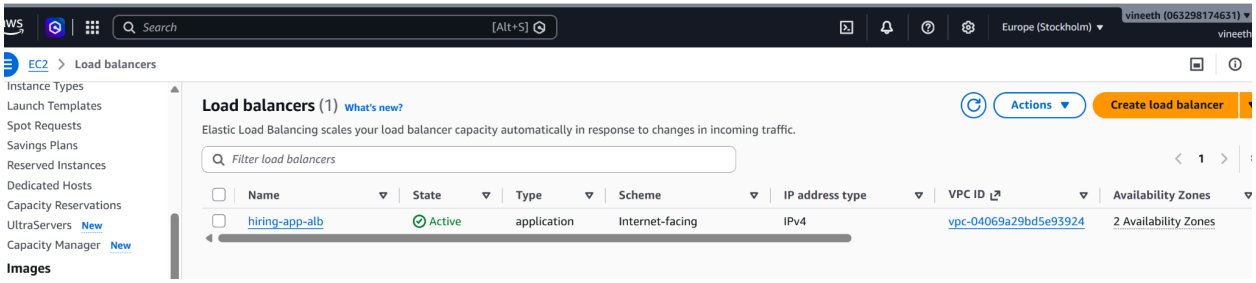
We created:

`hiring-app-alb`

Configuration:

- Type: Application Load Balancer
- Scheme: Internet-facing
- Listener protocol: HTTP
- Listener port: `80`

Purpose: The ALB gives users one public entry point to access the application.




## Step 7 — Create the target group

We created:

`hiring-app-tggg`

Configuration:

- Target type: IP
- Protocol: HTTP
- Application container port: `8080`
- Health check path: `/hiring/`

Purpose: The target group registers the ECS task IP addresses and checks whether the application responds correctly.

Initially, the health check used `/`, which returned HTTP 404. We changed the path to `/hiring/`, after which all four targets became healthy.

## Step 8 — Connect the load balancer to the target group

We configured the ALB listener on port `80` to forward requests to the target group.

The traffic flow is:

User's browser

HTTP request

Application Load Balancer

Port 80

Target group

Health checks and traffic routing

ECS Fargate tasks

Application container on port 8080

### Step 9 — Verify the application

We opened the application through the ALB URL:

Open the hiring application

The browser displayed:

Welcome to Techie Horizon

This confirmed that the browser could reach the application through the load balancer.

### Step 10 — Check task health and troubleshoot

We checked the target group and confirmed:

- 4 registered targets
- 4 healthy targets
- 0 unhealthy targets

We also checked ECS deployment events. The service failed to launch all 6 tasks because the account's Fargate On-Demand vCPU quota was 4.

A quota increase request for 8 vCPUs was already open.

Purpose: Monitoring task health and service events helps us identify deployment problems and understand why tasks cannot start.

## 6. Final result

| Item | Final status |
| --- | --- |
| ECS cluster | Active |
| ECS service | Active |
| Task definition | `hiring-app-task:2` |

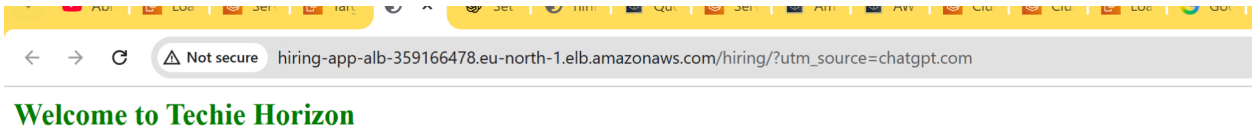
| | |
|---|---|
| Running tasks | 4 |
| Desired tasks | 4 |
| Load balancer | `hiring-app-alb` |
| Target group | `hiring-app-tggg` |
| Healthy targets | 4 |
| Application access | Successful |
| Six-task scaling | Pending quota increase |

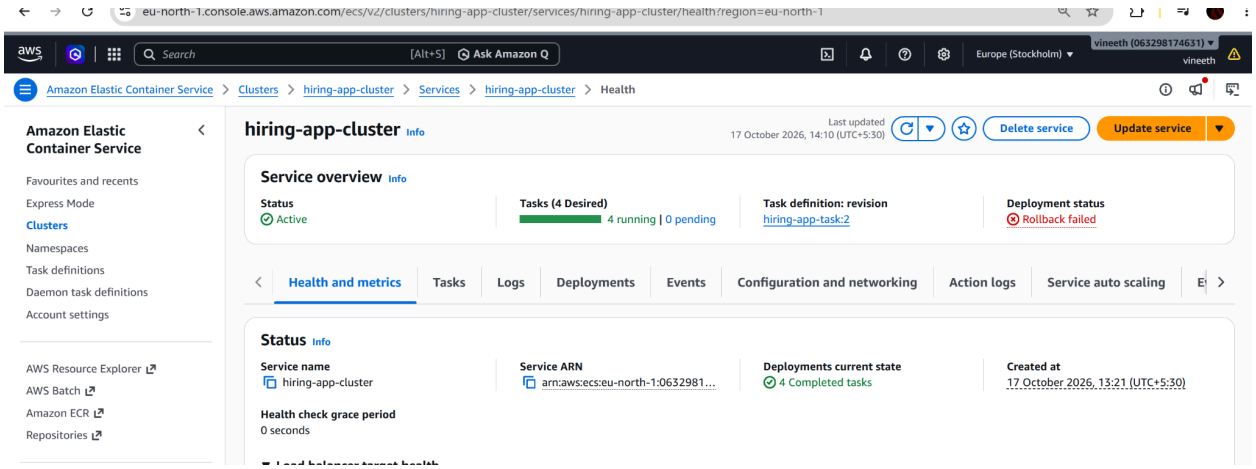
# 7. Conclusion

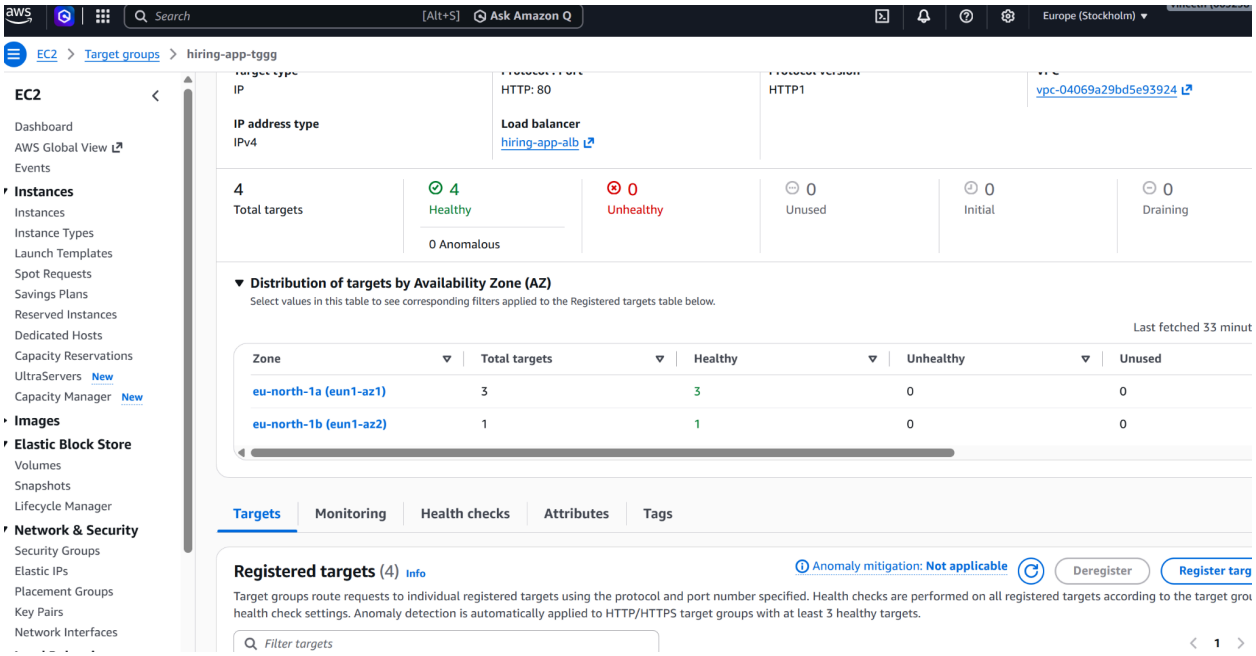
We successfully deployed the Techie Horizon hiring application using Amazon ECS and AWS Fargate. We created an Application Load Balancer, connected it to a target group, configured health checks, and accessed the application through a browser.

The application is currently running with four healthy tasks across the configured subnets. Scaling to six tasks remains pending because AWS must increase the Fargate vCPU quota.

What I learned: How to deploy a containerized application on ECS, configure Fargate networking, connect an ALB to a target group, troubleshoot health checks, and understand AWS service quotas.




eu-north-1.console.aws.amazon.com/ecs/v2/clusters/hiring-app-cluster/services/hiring-app-cluster/health?region=eu-north-1
Search
[Alt+S]
Ask Amazon Q
Europe (Stockholm)
vineeth (063298174631)
vineeth
Amazon Elastic Container Service > Clusters > hiring-app-cluster > Services > hiring-app-cluster > Health
Amazon Elastic Container Service
Favourites and recents
Express Mode
Clusters
Namespaces
Task definitions
Daemon task definitions
Account settings
AWS Resource Explorer
AWS Batch
Amazon ECR
Repositories
hiring-app-cluster Info
Last updated
17 October 2026, 14:10 (UTC+5:30)
Delete service
Update service
Service overview Info
Status
Active
Tasks (4 Desired)
4 running | 0 pending
Task definition: revision
hiring-app-task:2
Deployment status
Rollback failed
Health and metrics
Tasks
Logs
Deployments
Events
Configuration and networking
Action logs
Service auto scaling
Status Info
Service name
hiring-app-cluster
Service ARN
arn:aws:ecs:eu-north-1:0632981...
Deployments current state
4 Completed tasks
Created at
17 October 2026, 13:21 (UTC+5:30)
Health check grace period
0 seconds


Search
[Alt+S]
Ask Amazon Q
Europe (Stockholm)
EC2 > Target groups > hiring-app-tggg
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
IP
HTTP: 80
HTTP1
vpc-04069a29bd5e93924
IP address type
IPv4
Load balancer
hiring-app-alb
4
Total targets
4
Healthy
0 Anomalous
0
Unhealthy
0
Unused
0
Initial
0
Draining
Distribution of targets by Availability Zone (AZ)
Select values in this table to see corresponding filters applied to the Registered targets table below.
Last fetched 33 minut
Zone
Total targets
Healthy
Unhealthy
Unused
eu-north-1a (eun1-az1)
3
3
0
0
eu-north-1b (eun1-az2)
1
1
0
0
Targets
Monitoring
Health checks
Attributes
Tags
Registered targets (4) Info
Anomaly mitigation: Not applicable
Deregister
Register targ
Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target grou
health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.
Filter targets
1